# Pharma Cold Chain Data Warehouse and Analytics Platform

Phase 1, Milestone 1: Progress Report

Course: Intelligent Data Architectures

Prepared by: Narin Nadeem | Date: 11 November 2026

## 1. Summary

Phase 1 builds the Sales and Demand Intelligence (SDI) mart from three source files: master data (63 medicines), pharmacy sales (500,000 rows) and hospital demand (27,216 rows). The target for Milestone 1 is an automated pipeline from raw files to the analytical query layer, orchestrated by Apache Airflow.

| Component | Status |
|---|---|
| Environment: Docker, Airflow 3.4.0, PostgreSQL 16, DuckDB | Done |
| Synthetic data generator (continues IDs, advances dates into 2026) | Done |
| Ingestion to PostgreSQL staging with validation and reject table | Done (525,000 sales and 29,937 hospital rows loaded) |
| Star schema in DuckDB with SCD Type 2 on Dim_Medicine | Done |
| Analytical queries with timing log | Done (13 to 16 ms per query) |
| Pipeline observability: batch metrics table | Done |
| Airflow DAG running end to end from the UI | **Not working yet**: see section 7 |
| File-watcher trigger | Not built yet |

**Where I am stuck.** Each script works on its own and the first DAG task succeeds under *airflow tasks test*, but when the DAG is triggered from the Airflow UI the first task fails almost immediately with an empty log. Details and next actions are in section 7.

## 2. Architecture

| Stage | What it does | Tool / script |
|---|---|---|
| Generate | Creates new sales and hospital rows as timestamped files in data/landing | oversample.py |
| Ingest + validate | Empties staging, validates rows, loads raw and landing files with COPY, logs metrics | ingest_staging.py, PostgreSQL |
| Model | Builds dimensions and facts (star schema) from staging | build_datamart.py, DuckDB |

| Stage | What it does | Tool / script |
|---|---|---|
| Query | Runs three analytical queries and logs execution time | run_queries.py |
| Orchestrate | DAG phase1_sdi_pipeline: four tasks in sequence | Apache Airflow (CeleryExecutor) |

## 3. Data generation logic and assumptions

- The generator samples existing rows and gives them **new IDs**. It reads the highest ID from both data/raw and data/landing, so IDs continue (SALE-000525001 onward) and never repeat across batches.
- **Dates move forward** into 2026 (sales from 2026-01-01; hospital demand by month).
- Only valid MedicineIDs from the master file are used; quantities and amounts are recomputed so totals stay consistent.
- The first batch (before the DAG version) added 25,000 sales rows and 2,721 hospital rows. The DAG version creates 2 batches per run: 10,000 sales rows each, and 756 hospital rows each (63 medicines x 12 hospitals = one month of demand).
- Assumption: synthetic data keeps the distribution of the historical data. Realistic seasonality is not modelled.

## 4. Ingestion, validation and observability

Before every load the three staging tables are emptied, so a rerun replaces the data instead of duplicating it. Rows that fail a rule are written to **staging.stg_rejected_records** (source file, record ID, reason, raw payload) and are not loaded.

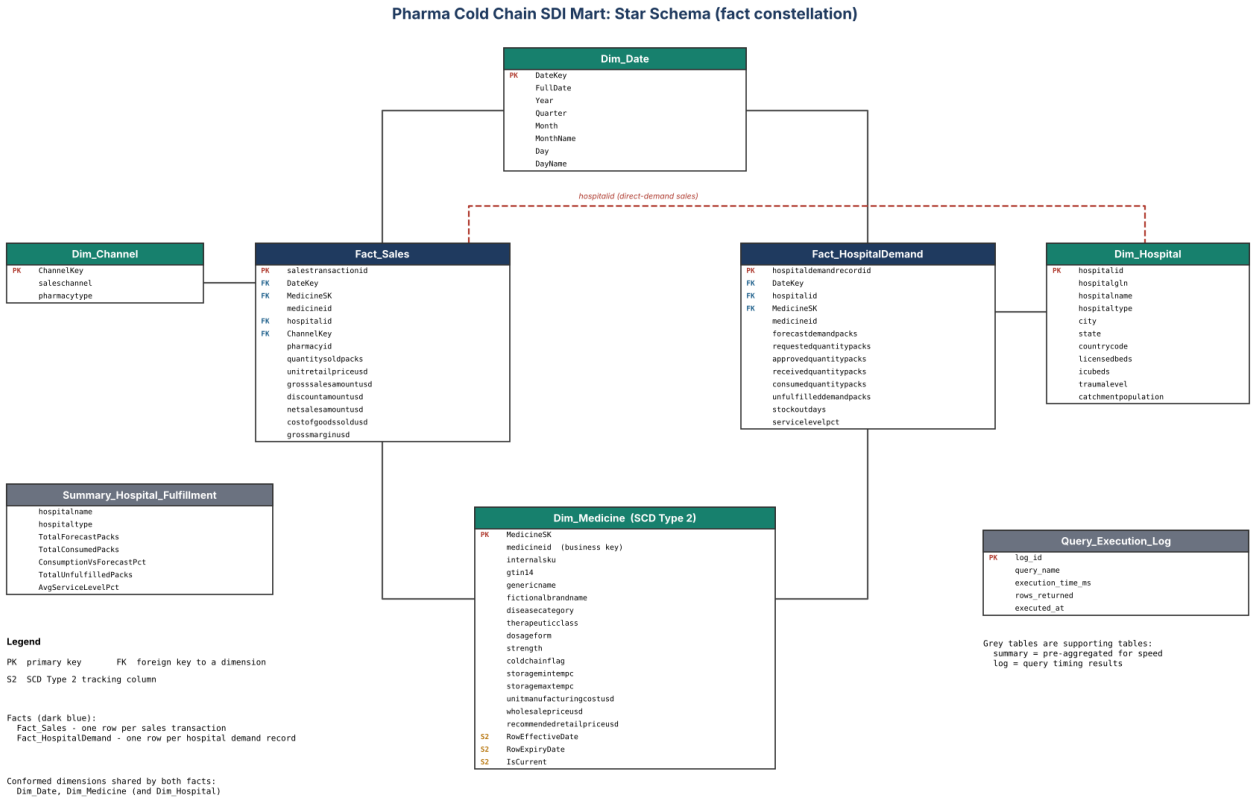
| Table | Validation rules |
|---|---|
| Master data | MedicineID not null; WholesalePriceUSD greater than 0 |
| Pharmacy sales | SalesTransactionID not null; UnitRetailPriceUSD greater than 0 |
| Hospital demand | HospitalDemandRecordID not null; ConsumedQuantityPacks not negative |

Load metrics from **staging.pipeline_metrics_batch** (run executed inside the Airflow container, all SUCCESS):

| Source file | Rows loaded | Seconds |
|---|---|---|
| 01_master_data.csv | 63 | 0.10 |
| 05_pharmacy_sales_data.csv | 500,000 | 28.57 |
| incremental_sales_20261110_165028.csv | 25,000 | 1.10 |
| 06_hospital_demand_data.csv | 27,216 | 0.93 |
| incremental_hosp_20261110_165028.csv | 2,721 | 0.11 |

Staging totals after the load: **525,000** sales rows, **29,937** hospital demand rows, **63** master rows. No rejection messages were printed during this run.

# 5. Star schema (data mart)




| Table | Type | Rows | Notes |
|---|---|---|---|
| Fact_Sales | Fact | 525,000 | One row per sales transaction; quantity, revenue, discount, cost, margin |
| Fact_HospitalDemand | Fact | 29,937 | Forecast, requested, approved, received, consumed, unfulfilled demand, stockout days, service level |
| Dim_Medicine | Dimension | 64 | SCD Type 2 (RowEffectiveDate, RowExpiryDate, IsCurrent); 63 medicines plus one history row |
| Dim_Hospital | Dimension | 12 | Type, location, beds, ICU beds, trauma level, catchment population |
| Dim_Channel | Dimension | 9 | Sales channel combined with pharmacy type |
| Dim_Date | Dimension | 1,127 | Day, month, quarter, year |
| Summary_Hospital_Fulfillment | Supporting | - | Pre-aggregated table for the hospital fulfillment query |

# 6. Query layer and storage

| Query | Execution time |
|---|---|
| Q1. Hospital forecast vs actual consumption and unfulfilled demand | 15.64 ms |
| Q2. Channel revenue split and gross margin | 14.28 ms |
| Q3. Cold chain velocity: fast-moving vs slow-moving medicines | 13.03 ms |

The DuckDB file is 26.76 MB. Each execution is recorded in the table **Query_Execution_Log**. The summary table is the optimization for the hospital query.

## 7. Orchestration: current problem

The DAG **phase1_sdi_pipeline** has four tasks: synthesize_incremental_batches, stage_and_validate_postgres, transform_duckdb_star_schema, run_analytical_and_storage_queries.

- Triggered from the Airflow UI, the first task fails on both attempts almost immediately. The task has no start time and its log file is empty. The other three tasks show Upstream Failed. The run takes about one minute, which is the one-minute retry delay.
- Running the same task with *airflow tasks test* inside the worker container succeeds and writes the four batch files, so the generator code and the container paths are correct.
- The scripts also run correctly on their own (generator, ingestion, star schema build and queries).
- Working theory: the problem is in how the Celery worker starts tasks, not in the scripts. Next checks: worker and scheduler logs, and a full run with *airflow dags test*.

## 8. Infrastructure problems solved

| Problem | Cause | Fix |
|---|---|---|
| Docker Desktop crashed (containerd SIGBUS) | C: drive had 0.2 GB free | Freed space and moved Docker's disk image to D: |
| Mart build killed inside Docker | Docker memory limit of 3.65 GB | Set memory=5GB in .wslconfig (limit now 4.8 GiB) |
| Airflow containers unhealthy | Slow package install at startup | Waited for install; packages: duckdb, pandas, psycopg2, openpyxl |
| Duplicate IDs and double loads | Generator ignored landing; ingestion appended | IDs continue across raw and landing; staging emptied before each load |

## 9. Remaining work and questions

- Fix the DAG so it runs green from the UI and take the screenshot; run it a second time for the refresh.
- Add a file-watcher (or agree that Airflow scheduling is acceptable).
- Write the 1 to 2 page note on data generation (content is in section 3).
- Milestone 2: dashboards for ingestion, storage and query metrics.
- **Question for sir:** what should the Excel deliverable for Phase 1 contain?